

OOAD Ch10: Model-Based Software Engineering 與 UML

主題重點：軟體工程中為何需要建模、UML 的起源與用途、物件導向分析與設計的核心觀念，以及各種 UML diagrams 的使用情境。

軟體工程與 OOAD 的學習挑戰

1. 學習門檻與實務經驗的重要性

- 老師一開始強調，物件導向與軟體工程其實很難教。
- 原因不是觀念本身完全無法理解，而是學生通常缺乏：
 - 大型系統開發經驗
 - 長時間維護程式碼的痛苦經驗
 - 多年實際 coding 與 debugging 的背景

老師用 MBA 做比喻：

- 很多人去念 MBA 時，並沒有真正管理過公司
- 所以課程會大量使用 **case studies** 來訓練學生的判斷力
- 軟體工程也是一樣，必須透過大量案例反覆「洗腦」，才能逐漸理解哪些設計是好的、哪些做法未來會出事

老師也提到像 CMU 的軟體工程碩士，會更希望學生本身已經有足夠實務經驗，尤其是曾經被維護爛系統折磨過的人，這樣學起來才會真正有感。

2. 軟體架構師的迷思

- 在建築或土木工程裡，建築師畫好圖之後，工人可以按圖施工。
- 但在軟體產業，這套模式通常沒有那麼單純。
- 老師形容程式設計師是一種「很詭異的人」：有時候寫著寫著就會偏離原本設計，開始做自己的事。

因此：

- 不能期待自己學完 OOAD 後，未來工作只負責畫 UML 圖
- 卻完全不需要寫程式或理解程式細節

老師的意思很明確：

真正懂分析與設計的人，不能脫離 coding。

3. 是不是所有東西都要物件導向？

老師的答案是：不一定。

- 物件導向分析主要是拿來架構系統
- 它不是專門用來推導演算法
- 也不是為了極致效能最佳化而存在

如果你今天只是：

- 在實驗室做一個一兩千行的演算法驗證
- 想測某種方法的速度或效果

那麼硬把所有東西都包成很完整的 OO 結構，可能只是在浪費時間。

老師區分了兩種情境：

- 實驗室專案**：偏重演算法與驗證
- 商業產品**：偏重演化、維護與擴充

而物件導向最主要的價值，是在第二種情境。

Model-Based Software Engineering

1. 為什麼需要 Modeling？

老師認為，任何高複雜度工程都不可能直接跳下去做。

他舉了幾個例子：

- 航太工程會先做模型，放進風洞 (Wind Tunnel) 測試
- 電機電子工程在做昂貴晶片前，會先用 VHDL 描述邏輯並做大量 simulation

反觀軟體界最常見的錯誤則是：

有了 idea 就直接開始 coding。

老師認為這正是許多軟體品質低落、大家對軟體缺乏信心的重要原因之一，也就是過去所謂的 **software crisis**。

2. 正確的軟體工程流程

課堂上強調的標準流程是：

Idea -> Requirement and Spec -> Modeling and Design -> Coding

也就是說：

- 先有想法
- 再整理需求與規格
- 再做建模與設計

- 最後才寫程式

這和很多學生習慣的：

Idea -> Coding -> 崩潰 -> 重寫

是完全不同的思維。

3. 建模的目的

建模最重要的價值之一，是：

Delay Implementation

也就是：先不要急著寫 **code**。

老師強調，建模能幫助團隊在實作前：

- 先看出潛在設計瑕疵
- 強迫自己理解需求細節
- 避免「摸石頭過河」式開發
- 減少後續的時間與金錢浪費

即使最後工程師實作得不夠好，甚至系統還是會當，但至少：

- 你的大架構是清楚的
- 你的概念完整性還在
- 系統不會完全失控

這就是老師說的：

分析與建模至少能讓大架構 maintained in good shape。



模型的種類與用途

老師指出，軟體開發中其實常常不自覺在用各種模型，例如：

- 流程圖
- 資料流圖
- 各種草圖

只是很多都不是標準化表示法。

依照嚴謹度分類

1. 不嚴謹的模型 (Not rigid)

例如：

- [Class Diagram](#)
- [ERD](#)

這些主要是：

- 幫助視覺化
- 幫助溝通
- 不能直接執行

2. 可執行的模型 (Executable)

例如：

- [State Chart Diagram](#)

這些模型可以拿來做模擬。

3. 嚴謹的模型 (Rigid)

- 可交給工具做更嚴格的驗證 (verification)
- 可以帶有比較接近數學證明的能力



UML：Unified Modeling Language

1. UML 的起源

老師提到，過去物件導向分析領域有著名的三位大師 (**The 3 Amigos**)：

- [Grady Booch](#)
- [Jim Rumbaugh](#)
- [Ivar Jacobson](#)

他們各自有自己的分析方法論，而且彼此都很強勢、不太願意妥協。

後來在 1994 年，Booch 和 Rumbaugh 先合流，接著在 1996 年推出了 **UML 1.0**，希望建立一套統一的圖示語言。

2. UML 是工具，不是方法論

老師特別提醒：

學會畫 UML，不代表你懂 OOAD。

就像：

- 技工看得懂施工圖
- 不代表他會做建築設計

UML 的角色比較像是：

- 標準化的圖形語言
- 幫助你表達分析設計結果
- 不是直接教你怎麼分析的方法

為什麼要用 UML？

1. 作為溝通的共同語言

軟體開發最核心的難題之一就是：**communication**。

老師認為：

- 只用自然語言描述系統，不夠精準
- 直接看 code，又太陷入細節

UML 剛好位於中間：

- 不會太抽象
- 也不會太底層

也就是老師說的：

- 不會「見樹不見林」
- 也不會「見林不見樹」

2. 作為系統的說明文件

UML 可以像大樓的管線圖一樣，留下系統藍圖。
未來系統維護時，這種藍圖很有價值。

3. 作為程式碼模板

透過一些 CASE tools，例如：

- [Rational Rose](#)
- [ArgoUML](#)

可以把 UML 模型轉成程式碼模板（template），再交給工程師補細節。

UML 的應用時機

1. 正向工程（Forward Engineering）

Specs -> Modeling -> Coding

這是老師認為比較正統、比較能防患未然的作法。

2. 逆向工程（Backward Engineering）

Code -> UML

也就是 code 都寫完之後，才反過來用工具生出 UML 圖。

這樣做仍然有幫助，但本質上比較像：

- 補文件
- 亡羊補牢

而不是前期真正做好分析。

物件導向分析與設計的核心精髓

1. 對付未來的變化

老師認為，分析最根本的目的不是為了畫圖本身，而是：

讓程式能夠承受未來的變化。

因為商業產品最麻煩的地方就在於：

- 規則會改
- UI 會改
- 技術平台會改
- 客戶需求會改

唯一不變的，就是一直在變。

2. Technical Debt

如果沒有先做設計與分析，只為了趕快把功能寫出來，就會留下大量 **technical debt**。

短期也許看起來比較快，長期則會讓：

- 維護困難
- 擴充困難
- 一改就壞

老師也提到，雖然 [Agile](#) 不主張百分之百全部分析完再開始，但也不代表可以完全不分析。

3. 好的分析不一定帶來好的效能

這是一個很重要的提醒。

老師說，通常反而相反：

- 結構良好的 OO 軟體
- 會有更多物件建立
- 會有更多方法呼叫
- 這些都可能降低效能

所以在某些情況下：

- 為了維持結構，會犧牲一點效能
- 為了追求效能，甚至可能把原本拆開的 class 再合併回去

Jacobson 的穩定度分層觀念

老師提到 [Jacobson](#) 對系統穩定度的一種理解方式，從最穩定到最易變大致可以看成：

1. **Object Classes / Entity**
2. **Object Attributes**
3. **Object Behavior**
4. **User Interface / Outputs**

也就是說：

- 最底層的 entity 比較穩定
- 行為比資料更容易改
- UI 與輸出通常變得最快

成功心法

老師強調一個重要心法：

透過 [Polymorphism](#) 等技術，把會變的東西盡量往上推。

讓底層穩定，讓變動集中在高層。

Control Objects 與 Abstract Classes

1. Control Objects

當你發現某個 method：

- 需要跨很多物件的資料
- 邏輯上又不屬於任何單一 entity
- 放在現有 class 內都很奇怪

就可以考慮建立 **control object** 把它包起來。

這和 [Ch9](#) 介紹的 Jacobson 三分類是呼應的。

2. Abstract Classes

分析階段常常會引入 **abstract class**，目的包括：

- 共用屬性
- 共用行為
- 為繼承與多型鋪路

有時這種 superclass 並不是自然界真的有對應物，而是為了設計而人工抽象出來的，因此老師稱它可能是一種：

- **Artificial Superclass**

避免「中毒太深」

老師提醒一個很有趣但很真實的現象：

- 如果你學了多型之後
- 看到所有 [switch](#) 都想把它改寫成 polymorphism
- 那就叫做「中毒太深」

也就是說，設計不是宗教。

如果某個 [switch](#)：

- 未來根本不會擴充
- 邏輯很單純
- 保留起來更清楚

那麼它就不一定非得被物件導向化。

UML 各類圖表簡介

因為軟體系統有很多不同面向，所以不能只靠單一圖表表達全部資訊。UML 提供了多種圖來描述不同觀點。

1. Use Case Diagram

- 用來描述系統需求與使用者互動
- 會出現像火柴人的 **Actor**
- Actor 指向系統功能 (use cases)

這種圖的重點不在寫作細節，而在：

- 誰在使用系統
- 會做哪些事

2. Class Diagram

- 用來描述系統的靜態資料結構
- 一個 class 方塊通常分三層：
 1. 名稱
 2. 屬性
 3. 操作 / 方法

圖上還會標示：

- multiplicity (如 **1..***, **0..***)
- association
- generalization / inheritance
- aggregation
- navigability

這是 OOAD 最常見、也最核心的圖之一。

3. Sequence Diagram 與 Collaboration Diagram

Sequence Diagram

- 用來描述執行過程中物件間的動態互動
- 最上方是物件
- 往下虛線是 **lifeline**
- 長條方塊是 **activation bar**
- 橫向前頭是 message / method call

它還可以表現：

- iteration
- condition
- object creation
- deletion

Collaboration Diagram

- 本質上與 sequence diagram 表達的是相近資訊
- 只是更強調物件之間的關聯與訊息順序
- 順序通常用像 **1.1.1** 這樣的編號表示

4. State Chart Diagram

- 建立在 finite-state machine 概念上
- 用來描述單一物件或系統的狀態變化

包含：

- initial state
- final state
- event
- guard condition
- activity / action

這類圖在描述反應式系統、事件驅動行為時特別有用。

5. Activity Diagram

- 外觀很像傳統 flow chart
- 但特別擅長描述：
 - synchronization
 - concurrency
 - parallel flows

老師形容它像：

- 烤肉架
- 游泳池

裡面的區塊叫做 **swimlane**，用來區分：

- 不同角色
- 不同子系統
- 不同責任區

也能表達：

- fork
- join

例如 ATM 系統中：

- ATM machine 與銀行伺服器可同時進行檢查
- 等到雙方都完成後，再進入下一步

6. Package Diagram 與 Object Diagram

當系統規模變大時，需要更高層次的模組視角。

Package Diagram

- 用來顯示 modules / packages 之間的 dependency
- 如果 UI 指向 Ordering，代表：
 - UI 依賴 Ordering
 - Ordering 改變時，UI 可能也得修改或重新編譯

Object Diagram

- 比 class diagram 更接近某個時間點下的具體物件實例關係

7. Component Diagram 與 Deployment Diagram

這兩種圖偏向更高階的系統架構表示。

Component Diagram

在新版 UML 中常看到：

- **lollipop**：代表某元件提供的 service
- **socket / delegation connector**：代表某元件依賴或使用某服務

Deployment Diagram

- 描述實際硬體節點 (node)
- 例如：PC、Server、Database Server
- 以及節點間使用的通訊方式 (例如 [TCP/IP](#))

✔ 本章核心觀念總結

1. OOAD 與軟體工程難學的原因，在於它高度依賴實務與維護經驗。
2. **Modeling** 的目的不是拖時間，而是延遲實作、提早發現設計問題。
3. 標準流程應是： [Idea -> Spec -> Modeling -> Coding](#)。
4. **UML** 是標準化圖示語言，但它不是方法論本身。
5. 分析與設計的真正目的是面對未來變化、降低 **technical debt**。
6. 好結構不一定帶來最好效能，兩者常常需要取捨。
7. 透過多型與抽象化，應盡量讓不穩定的東西往上層集中。
8. 不要過度物件導向化，設計應服務需求，而不是為了炫技。
9. UML 不同圖表各自描述系統不同面向：需求、靜態結構、動態互動、狀態、活動流程、模組依賴與部署架構。

🔗 Related Notes

- [📖 School/OOAD_Object-Oriented Analysis and Design](#)
- [School/OOAD/CH9_How_to_Find_Classes_and_Objects](#)
- [School/OOAD/CH8_OOAD_ERD_and_Normalization](#)